

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
Санкт-Петербургский национальный исследовательский университет ИТМО
Мегафакультет трансляционных информационных технологий

Факультет прикладной информатики

Лабораторная работа №6

По дисциплине «Проектирование и реализация баз данных»

Выполнил студент группы №К3240,
Гаенко Анастасия Константиновна

Преподаватель:
Говорова Марина Михайловна



Санкт-Петербург
2026

Цель работы:

Овладеть практическими навыками работы с CRUD-операциями, вложенными объектами в коллекциях базы данных MongoDB, агрегированными запросами, изменением данных, использованием ссылок и индексов в MongoDB.

Практическое задание:

1. Выполнить CRUD-операции в СУБД MongoDB: вставку, выборку, изменение и удаление документов.
2. Освоить работу с запросами к вложенным объектам и массивам.
3. Использовать курсоры и агрегированные запросы для обработки данных.
4. Реализовать обновление документов с применением операторов MongoDB.
5. Изучить способы организации ссылок между коллекциями.
6. Настроить и исследовать индексы в MongoDB, а также проанализировать план выполнения запросов.

№2.1.1

1. *Создайте базу данных learn.*
2. *Заполните коллекцию единорогов unicorns:*

```
switched to db learn
> db.unicorns.insertMany([
  {
    name: 'Horny',
    loves: ['carrot', 'papaya'],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: ['carrot', 'grape'],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: ['energon', 'redbull'],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    name: 'Rooodoodles',
    loves: ['apple'],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    name: 'Solnara',
    loves: ['apple', 'carrot', 'chocolate'],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
```

1. Используя второй способ, вставьте в коллекцию единорогов документ
2. Проверьте содержимое коллекции с помощью метода *find*.

```

    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6a18288a6c427cab34d9bef0'),
      '1': ObjectId('6a18288a6c427cab34d9bef1'),
      '2': ObjectId('6a18288a6c427cab34d9bef2'),
      '3': ObjectId('6a18288a6c427cab34d9bef3'),
      '4': ObjectId('6a18288a6c427cab34d9bef4'),
      '5': ObjectId('6a18288a6c427cab34d9bef5'),
      '6': ObjectId('6a18288a6c427cab34d9bef6'),
      '7': ObjectId('6a18288a6c427cab34d9bef7'),
      '8': ObjectId('6a18288a6c427cab34d9bef8'),
      '9': ObjectId('6a18288a6c427cab34d9bef9'),
      '10': ObjectId('6a18288a6c427cab34d9befa')
    }
  }
}
let unicorn = {
  name: 'Dunx',
  loves: ['grape', 'watermelon'],
  weight: 704,
  gender: 'm',
  vampires: 165
}
db.unicorns.insertOne(unicorn)
{
  acknowledged: true,
  insertedId: ObjectId('6a1828f56c427cab34d9befb')
}
db.unicorns.find().pretty()
{
  _id: ObjectId('6a18288a6c427cab34d9bef0'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}

```

№2.2.1

1. Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

```
}
> db.unicorns.find({gender: 'm'}).sort({name: 1})
< {
  _id: ObjectId('6a1828f56c427cab34d9befb'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef0'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: -63
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef6'),
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef9'),
  name: 'Pilot',
```

```

> db.unicorns.find({gender: 'f'})
.sort({name: 1})
.limit(3)
< {
  _id: ObjectId('6a18288a6c427cab34d9bef1'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef5'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 733,
  gender: 'f',
  vampires: 40
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef8'),
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}
> db.unicorns.find({
  gender: 'f',
  loves: 'carrot'

```

2. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций *findOne* и *limit*.

```

db.unicorns.findOne({
  gender: 'f',
  loves: 'carrot'
})
{
  _id: ObjectId('6a18288a6c427cab34d9bef1'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
db.unicorns.find({
  gender: 'f',
  loves: 'carrot'
}).limit(1)
{
  _id: ObjectId('6a18288a6c427cab34d9bef1'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}

```

№2.2.2

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
> db.unicorns.find(
  { gender: 'm' },
  { loves: 0, gender: 0 }
)
< {
  _id: ObjectId('6a18288a6c427cab34d9bef0'),
  name: 'Horny',
  weight: 600,
  vampires: 63
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef2'),
  name: 'Unicrom',
  weight: 984,
  vampires: 182
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef3'),
  name: 'Rooooooodles',
  weight: 575,
  vampires: 99
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef6'),
  name: 'Kenny',
  weight: 690,
  vampires: 39
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef7'),
  name: 'Raleigh',
  weight: 421,
  vampires: 2
}
{
```

№ 2.2.3

Вывести список единорогов в обратном порядке добавления.


```
> db.unicorns.find().sort({ $natural: -1 })
< {
  _id: ObjectId('6a1828f56c427cab34d9befb'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a18288a6c427cab34d9befa'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef9'),
  name: 'Pilot',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 650,
  gender: 'm',
  vampires: 54
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef8'),
  name: 'Leia',
  loves: [
```

№2.1.4

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
db.unicorns.find(
  {},
  { _id: 0, name: 1, loves: { $slice: 1 } }
)
[
  {
    name: 'Horny',
    loves: [
      'carrot'
    ]
  },
  {
    name: 'Aurora',
    loves: [
      'carrot'
    ]
  },
  {
    name: 'Unicrom',
    loves: [
      'energon'
    ]
  },
  {
    name: 'Rooooooodles',
    loves: [
      'apple'
    ]
  },
  {
    name: 'Solnara',
    loves: [
      'apple'
    ]
  },
  {
    name: 'Ayna',
    loves: [
```

№2.3.1

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```

> db.unicorns.find(
  {
    gender: 'f',
    weight: { $gte: 500, $lte: 700 }
  },
  {
    _id: 0
  }
)
{
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}
{
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],

```

№2.3.2

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
> db.unicorns.find(
  {
    gender: 'm',
    weight: { $gte: 500 },
    loves: { $all: ['grape', 'lemon'] }
  },
  {
    _id: 0
  }
)
< {
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}
learn >
```

№2.3.3

Найти всех единорогов, не имеющих ключ `vampires`.

```
}  
> db.unicorns.find({  
  vampires: { $exists: false }  
})  
< {  
  _id: ObjectId('6a18288a6c427cab34d9befa'),  
  name: 'Nimue',  
  loves: [  
    'grape',  
    'carrot'  
  ],  
  weight: 540,  
  gender: 'f'  
}
```

№2.3.4

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
> db.unicorns.find(
  {
    gender: 'm'
  },
  {
    _id: 0,
    name: 1,
    loves: { $slice: 1 }
  }
).sort({ name: 1 })
< {
  name: 'Dunx',
  loves: [
    'grape'
  ]
}
{
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Kenny',
  loves: [
    'grape'
  ]
}
{
  name: 'Pilot',
  loves: [
    'apple'
  ]
}
{
  name: 'Raleigh',
  loves: [
    'apple'
  ]
}
```

№3.1.1

1. Создайте коллекцию towns, включающую следующие документы:

```

db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a18310acf05bf77623c23ba'),
    '1': ObjectId('6a18310acf05bf77623c23bb'),
    '2': ObjectId('6a18310acf05bf77623c23bc')
  }
}

```

2. Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```

> db.towns.find(
  { "mayor.party": "I" },
  { _id: 0, name: 1, mayor: 1 }
)
< {
  name: 'New York',
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}

```

3. Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```

> db.towns.find(
  { "mayor.party": { $exists: false } },
  { _id: 0, name: 1, mayor: 1 }
)
< {
  name: 'Punxsutawney',
  mayor: {
    name: 'Jim Wehrle'
  }
}
learn> |

```


№3.1.2

1. Сформировать функцию для вывода списка самцов единорогов.
2. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
3. Вывести результат, используя *forEach*.
4. Содержание коллекции единорогов *unicorns*:

```
> use learn
< switched to db learn
> function getMaleUnicorns() {
    return db.unicorns.find({ gender: "m" });
}

var cursor = getMaleUnicorns()
    .sort({ name: 1 })
    .limit(2)

cursor.forEach(function(obj) {
    print(obj.name)
})
< Dunx
< Horny
learn>
```

№3.2.1

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
< Horny
> db.unicorns.find({
  gender: "f",
  weight: { $gte: 500, $lte: 600 }
}).count()
< 2
```

№3.2.2

Вывести список предпочтений.

```
< 2
> db.unicorns.distinct("loves")
< [
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

№3.2.3

Посчитать количество особей единорогов обоих полов.

```
]
> db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      count: { $sum: 1 }
    }
  }
])
< {
  _id: 'm',
  count: 7
}
{
  _id: 'f',
  count: 5
}
>
learn>
```

№3.3.1

1. Выполнить команду:

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

2. Проверить содержимое коллекции *unicorns*.

```

    })
    ✖ ▶ TypeError: db.unicorns.save is not a function
    > db.unicorns.insertOne({
      name: 'Barny',
      loves: ['grape', 'apple'],
      weight: 530,
      gender: 'm',
      vampires: 132
    })
    < {
      acknowledged: true,
      insertedId: ObjectId('6a1841cf3ee7b2ac5aa6167c')
    }
    >
    >
    > db.unicorns.find({ name: 'Barny' })
    < {
      _id: ObjectId('6a1841cf3ee7b2ac5aa6167c'),
      name: 'Barny',
      loves: [
        'grape',
        'apple'
      ],
      weight: 530,
      gender: 'm',
      vampires: 132
    }
    learn> |

```

№3.3.2

1. Для самки единорога Ауна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции *unicorns*.

```
MONGOSH
db.unicorns.updateOne(
  { name: 'Ayna' },
  {
    $set: {
      weight: 800,
      vampires: 51
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
db.unicorns.find({ name: 'Ayna' })
{
  _id: ObjectId('6a18288a6c427cab34d9bef5'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 800,
  gender: 'f',
  vampires: 51
}
```

№3.3.3

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции *unicorns*

```
}  
> db.unicorns.updateOne(  
  { name: 'Raleigh' },  
  {  
    $set: {  
      loves: ['redbull']  
    }  
  }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0
```

№3.3.4

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.
2. Проверить содержимое коллекции *unicorns*.

```
    }  
  > db.unicorns.updateMany(  
    { gender: 'm' },  
    {  
      $inc: {  
        vampires: 5  
      }  
    }  
  )  
  < {  
    acknowledged: true,  
    insertedId: null,  
    matchedCount: 8,  
    modifiedCount: 8,  
    upsertedCount: 0  
  }
```

№3.3.5

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции *towns*.

```
db.towns.updateOne(
  { name: 'Portland' },
  {
    $unset: {
      "mayor.party": ""
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

№3.3.6

1. Изменить информацию о самце единорога *Pilot*: теперь он любит и шоколад.
2. Проверить содержимое коллекции *unicorns*.


```
> db.unicorns.updateOne(
  { name: 'Pilot' },
  {
    $push: {
      loves: 'chocolate'
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.unicorns.updateOne(
```

№3.3.7

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции *unicorns*.

```

}
db.unicorns.updateOne(
  { name: 'Aurora' },
  {
    $addToSet: {
      loves: {
        $each: ['sugar', 'lemon']
      }
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn >

```

№3.4.1

1. Создайте коллекцию *towns*, включающую следующие документы
2. Удалите документы с беспартийными мэрами.
3. Проверьте содержание коллекции.
4. Очистите коллекцию.
5. Просмотрите список доступных коллекций.

```

> use learn
< switched to db learn
> db.towns.deleteMany({
  "mayor.party": { $exists: false }
})
< {
  acknowledged: true,
  deletedCount: 2
}
> db.towns.find()
< {
  _id: ObjectId('6a18310acf05bf77623c23bb'),
  name: 'New York',
  population: 22200000,
  last_sensus: 2009-07-31T00:00:00.000Z,
  famous_for: [
    'status of liberty',
    'food'
  ],
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
> db.towns.deleteMany({})
< {
  acknowledged: true,
  deletedCount: 1
}
> show collections
< towns
  unicorns
learn>

```

№4.1.1

1. Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.
2. Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.

```

> use learn
< switched to db learn
> db.habitats.find()
< {
  _id: ObjectId('6a1844543ee7b2ac5aa6167d'),
  name: 'Forest'
}
{
  _id: ObjectId('6a1844543ee7b2ac5aa6167e'),
  name: 'Mountain'
}
{
  _id: ObjectId('6a1844543ee7b2ac5aa6167f'),
  name: 'Lake'
}
> db.unicorns.updateOne(
  { name: "Horny" },
  {
    $set: {
      habitat: ObjectId("6a1844543ee7b2ac5aa6167d")
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.unicorns.updateOne(
  { name: "Aurora" },
  {
    $set: {
      habitat: ObjectId("6a1844543ee7b2ac5aa6167d")
    }
  }
)
< {
  acknowledged: true

```

3. Проверьте содержание коллекции едиорогов.

```

>_MONGOSH
> db.unicorns.find(
  {},
  {
    name: 1,
    habitat: 1
  }
)
< {
  _id: ObjectId('6a18288a6c427cab34d9bef0'),
  name: 'Horny',
  habitat: ObjectId('6a1844543ee7b2ac5aa6167d')
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef1'),
  name: 'Aurora',
  habitat: ObjectId('6a1844543ee7b2ac5aa6167d')
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef2'),
  name: 'Unicrom'
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef3'),
  name: 'Roooooodles'
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef4'),
  name: 'Solnara'
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef5'),
  name: 'Ayna'
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef6'),
  name: 'Kenny'
}
{
  _id: ObjectId('6a18288a6c427cab34d9bef7'),
  name: 'Raleigh'
}

```

№4.2.1

1. Проверьте, можно ли задать для коллекции *unicorns* индекс для ключа *name* с флагом *unique*.

```

> db.unicorns.createIndex(
  { name: 1 },
  { unique: true }
)
< name_1
> db.unicorns.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
> db.unicorns.insertOne({
  name: "Horny"
})
✖ ▶ MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Horny" }

```

№4.3.1

1. Получите информацию о всех индексах коллекции *unicorns*.
2. Удалите все индексы, кроме индекса для идентификатора.
3. Попробуйте удалить индекс для идентификатора.

```

> db.unicorns.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
> db.unicorns.dropIndexes()
< {
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
> db.unicorns.dropIndex("_id_")
✖ ▶ MongoServerError[InvalidOptions]: cannot drop _id index
learn> |

```

№4.4.1

1. Создайте объемную коллекцию *numbers*, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```

2. Выберите последних четыре документа.

```
> use learn
< switched to db learn
> for (let i = 0; i < 100000; i++) {
    db.numbers.insertOne({ value: i })
}
< {
  acknowledged: true,
  insertedId: ObjectId('6a184732d77e145ef2f61cc4')
}
> db.numbers.find().sort({ value: -1 }).limit(4)
< {
  _id: ObjectId('6a184732d77e145ef2f61cc4'),
  value: 99999
}
{
  _id: ObjectId('6a184732d77e145ef2f61cc3'),
  value: 99998
}
{
  _id: ObjectId('6a184732d77e145ef2f61cc2'),
  value: 99997
}
{
  _id: ObjectId('6a184732d77e145ef2f61cc1'),
  value: 99996
}
```

3. Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)

```
executionTimeMillis: 57,
```

4. Создайте индекс для ключа `value`.

```
}
db.numbers.createIndex({ value: 1 })
value_1
```

5. Получите информацию о всех индексах коллекции `numbers`.

6. Выполните запрос 2.

```
> db.numbers.find()  
      .sort({ value: -1 })  
      .limit(4)  
      .explain("executionStats")  
< {
```

7. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

```
executionTimeMillis: 8,
```

8. Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

При сравнении планов выполнения запросов было установлено, что запрос без индекса выполняется менее эффективно, так как MongoDB просматривает всю коллекцию документов. После создания индекса по полю value запрос стал выполняться быстрее, поскольку поиск и сортировка выполняются с использованием индекса. Следовательно, более эффективным является запрос с индексом.

Вывод:

В ходе выполнения лабораторной работы были изучены основные возможности СУБД MongoDB. Были освоены операции создания и заполнения коллекций, выборки, изменения и удаления документов. Также были изучены методы работы с вложенными объектами, массивами и курсорами.

В процессе выполнения работы были использованы операторы фильтрации, сортировки и агрегирования данных, а также механизмы обновления документов с помощью операторов \$set, \$inc, \$push, \$unset и \$addToSet.

Кроме того, были рассмотрены способы организации связей между коллекциями с использованием ObjectId, а также работа с индексами и анализ производительности запросов с помощью explain().

В результате выполнения лабораторной работы были получены практические навыки работы с документо-ориентированной базой данных MongoDB и средствами обработки данных в NoSQL СУБД.